

RELATÓRIO TÉCNICO

ANÁLISE DE PERFORMANCE COMERCIAL E LOGÍSTICA EM OPERAÇÃO
GLOBAL DE VAREJO UTILIZANDO BUSINESS INTELLIGENCE E MACHINE
LEARNING

Paula da Fonseca Freire

Belo Horizonte

2026

SUMÁRIO

1. Introdução.....	3
1.1. Contexto.....	3
1.2. Objetivos.....	5
1.3. Público alvo.....	5
2. Modelo de Dados.....	7
2.1. Modelo Dimensional.....	7
2.2. Fatos e Dimensões.....	7
3. Integração, Tratamento e Carga de Dados.....	9
3.1. Fontes de Dados.....	9
3.2 Processos de Integração e Carga (ETL).....	11
3.2.1 Preparação do ambiente e estrutura local.....	13
3.2.2 Leitura do arquivo bruto e inspeção inicial.....	15
3.2.3 Padronização dos nomes das colunas.....	15
3.2.4 Tipagem dos campos e tratamento básico de qualidade.....	17
3.2.5 Criação de variáveis derivadas e geração da camada staging.....	19
3.2.6 Construção das dimensões.....	20
3.2.7 Construção da tabela fato e validações críticas.....	23
3.2.8 Exportação da camada curada e geração do relatório de qualidade.....	26
3.2.9 Carga no PostgreSQL (Neon).....	26
3.2.10 Validação analítica no banco.....	28
3.2.11 Consolidação técnica do processo.....	30
4. Camada de Apresentação.....	30
4.1 Dashboard.....	30
4.1.1 Visão Estratégica – Sales Performance.....	31
4.1.2 Visão Tática – Product Analysis.....	32
4.1.3 Visão Tática/Operacional – Customer & Order.....	33
4.1.4 Visão Operacional – Logistics.....	34
4.1.5 Filtros e Interatividade.....	35
4.2 Análises avançadas.....	35
5. Registros de Homologação.....	36
6. Conclusões.....	37
7. Links.....	38

1. Introdução

1.1. Contexto

Em ambientes corporativos modernos, especialmente no setor de varejo, a geração de dados ocorre de forma contínua, distribuída e em grande volume. Cada transação comercial, como um pedido realizado por um cliente, produz múltiplos registros que capturam diferentes aspectos da operação, incluindo informações sobre produtos, clientes, preços, descontos, custos logísticos, localização geográfica e prazos de entrega. Esses dados são fundamentais para o funcionamento operacional do negócio, pois sustentam atividades como faturamento, logística, controle de estoque e relacionamento com clientes.

No entanto, a forma como esses dados são originalmente estruturados não tem como objetivo principal a análise gerencial. Em geral, os sistemas transacionais priorizam a integridade e a consistência operacional, organizando os dados de maneira detalhada e orientada a eventos específicos. Como consequência, embora haja riqueza de informação, existe uma limitação significativa na capacidade de extrair conhecimento de forma rápida e confiável. Essa limitação se torna evidente quando gestores e analistas precisam responder a perguntas estratégicas, como a rentabilidade de determinadas categorias de produtos, o impacto dos descontos sobre o resultado financeiro ou a eficiência logística em diferentes regiões.

Esse cenário torna-se ainda mais complexo em operações de varejo com atuação global. Nesses contextos, os dados não apenas são volumosos, mas também heterogêneos, pois refletem diferentes mercados, regiões geográficas, comportamentos de consumo e estruturas logísticas. A análise isolada de indicadores como volume de vendas deixa de ser suficiente para representar a realidade do negócio. Torna-se necessário compreender a interação entre múltiplas variáveis, como lucro, margem, custo de envio, tempo de entrega, perfil dos clientes e sazonalidade das vendas.

Dessa forma, o problema central abordado neste projeto consiste na dificuldade de transformar dados transacionais brutos em informação analítica estruturada e confiável. Em um conjunto de dados com dezenas de milhares de registros, não basta identificar quanto foi vendido em determinado período. É necessário compreender onde estão as oportunidades de melhoria, quais

segmentos geram maior valor, quais fatores operacionais impactam negativamente a rentabilidade e como o desempenho evolui ao longo do tempo. Essa lacuna entre dados operacionais e visão gerencial compromete a tomada de decisão e reduz a eficiência estratégica da organização.

O conjunto de dados utilizado neste projeto representa um cenário típico desse contexto. Trata-se de um conjunto de dados públicos de varejo global, contendo dados históricos de 4 anos. A granularidade do conjunto de dados está no nível de item de pedido, ou seja, cada linha corresponde a um produto específico dentro de um pedido realizado. Essa característica é relevante, pois permite análises detalhadas e flexíveis, mas também aumenta a complexidade de organização dos dados para fins analíticos.

Entre os atributos disponíveis no conjunto de dados estão variáveis comerciais, como valor de vendas, lucro, quantidade e desconto; variáveis logísticas, como modo de envio, custo de entrega e prazo entre pedido e expedição; e variáveis contextuais, como cliente, produto, categoria, mercado, região, país, estado e cidade. Esse conjunto de informações oferece um potencial analítico significativo, mas, em seu formato original, não está preparado para responder de forma estruturada às necessidades de análise gerencial.

Do ponto de vista de aplicação, o projeto foi desenvolvido em contexto acadêmico, porém com estrutura e rigor compatíveis com um ambiente corporativo real. O cenário simulado envolve uma operação global de varejo que necessita melhorar sua capacidade de análise de desempenho comercial e logístico. Para isso, foram consideradas tecnologias amplamente utilizadas no mercado, como [Python](#) para manipulação e transformação de dados (ETL), [PostgreSQL](#) em *ambiente cloud* ([Neon](#)) para armazenamento analítico, [Power BI Desktop](#) para visualização e [scikit-learn](#) para análises de aprendizado de máquina (Machine Learning). Ferramentas auxiliares, como [ChartDB](#) e [Lucidchart](#), foram utilizadas para *representação visual da modelagem e dos fluxos de dados*, enquanto o [Google Docs](#) foi adotado para a documentação técnica do projeto. Adicionalmente, houve apoio do [ChatGPT](#) em *etapas pontuais do desenvolvimento*, especialmente na validação de abordagens, sugestões de melhorias estruturais e apoio na interpretação de determinados pontos técnicos do projeto.

Com base nesses elementos, o tema do projeto foi definido como [Análise de Performance Comercial e Logística em Operação Global de Varejo](#) utilizando

Business Intelligence e Machine Learning. Esse recorte direciona o foco do problema para a *compreensão integrada entre resultados comerciais e eficiência operacional*, considerando que ambos os aspectos são determinantes para a sustentabilidade do negócio.

1.2. Objetivos

No contexto apresentado, a organização possui como necessidade central melhorar a qualidade e a profundidade da análise sobre seu desempenho, de forma a sustentar decisões estratégicas mais consistentes e orientadas por dados. Em uma operação de varejo global, a dificuldade não está na disponibilidade da informação, mas na capacidade de interpretá-la de forma integrada e confiável.

Dessa forma, o objetivo deste projeto é *avaliar a performance comercial e logística de uma operação global de varejo, identificando os principais fatores que influenciam a rentabilidade e a eficiência operacional do negócio*.

Nesse sentido, o objetivo do projeto está direcionado à construção de uma visão analítica integrada que permita responder, de forma estruturada, questões como: quais mercados e regiões a operação apresenta melhor desempenho, quais categorias de produtos geram maior retorno financeiro, de que forma os descontos influenciam a rentabilidade e como os fatores logísticos impactam o resultado do negócio.

O foco do projeto está, portanto, na geração de uma visão integrada do negócio, capaz de relacionar indicadores comerciais, como vendas e quantidade, com indicadores financeiros e operacionais, como lucro, desconto e custo logístico. Essa abordagem permite identificar padrões de desempenho, pontos de eficiência e possíveis gargalos que afetam o resultado da operação.

1.3. Público alvo

A solução proposta neste projeto é direcionada a um público corporativo que depende de informação consolidada para acompanhamento de resultados e tomada de decisão, especialmente em contextos que envolvem operações comerciais e logísticas. O foco está em usuários que precisam interpretar indicadores de forma integrada, compreendendo não apenas os resultados obtidos, mas também os fatores que os influenciam.

Um dos principais públicos é composto por [gestores comerciais](#), que utilizam as análises para acompanhar o desempenho de vendas e direcionar estratégias. Esses usuários tendem a utilizar o painel para identificar quais mercados, regiões e categorias apresentam melhor resultado, avaliar a efetividade de políticas de desconto e entender em quais contextos o volume de vendas se converte, de fato, em rentabilidade. A análise permite apoiar decisões como priorização de mercados, ajustes comerciais e revisão de estratégias de preço.

Outro grupo relevante é formado por [gestores e analistas de operações e logística](#), que utilizam as informações para avaliar a eficiência operacional. Nesse caso, o uso das análises está relacionado à compreensão do impacto de variáveis como custo de envio, prazo de entrega e modo de transporte sobre o desempenho financeiro. Esses usuários conseguem identificar, por exemplo, em quais regiões o custo logístico compromete a margem ou em quais cenários a operação apresenta maior eficiência, permitindo ajustes operacionais mais direcionados.

Também fazem parte do público-alvo os [analistas de Business Intelligence](#), que utilizam a solução de forma mais exploratória. Esses profissionais tendem a aprofundar as análises, cruzando diferentes dimensões, validando consistência de indicadores e investigando padrões de comportamento nos dados. Seu uso está associado à geração de insights mais detalhados, identificação de anomalias e suporte técnico às áreas de negócio.

Além disso, a solução atende a [lideranças de planejamento e performance](#), que utilizam as análises para monitoramento de indicadores e acompanhamento da evolução do negócio ao longo do tempo. Para esse perfil, a principal aplicação está na identificação de tendências, variações de desempenho e possíveis desvios em relação ao esperado, apoiando a definição de planos de ação e o acompanhamento de resultados.

De forma geral, os usuários utilizam a solução para transformar dados em entendimento prático do negócio, seja para análise detalhada, acompanhamento de indicadores ou suporte à tomada de decisão. O uso das análises está diretamente relacionado à necessidade de compreender como diferentes variáveis (comerciais e logísticas) impactam o desempenho da operação, permitindo uma leitura mais completa e consistente dos resultados.

2. Modelo de Dados

2.1. Modelo Dimensional

Para este projeto, foi adotada uma modelagem dimensional no formato [Star Schema](#), organizada a partir de uma *tabela fato central* e *dimensões analíticas* que representam os principais eixos de leitura do negócio.

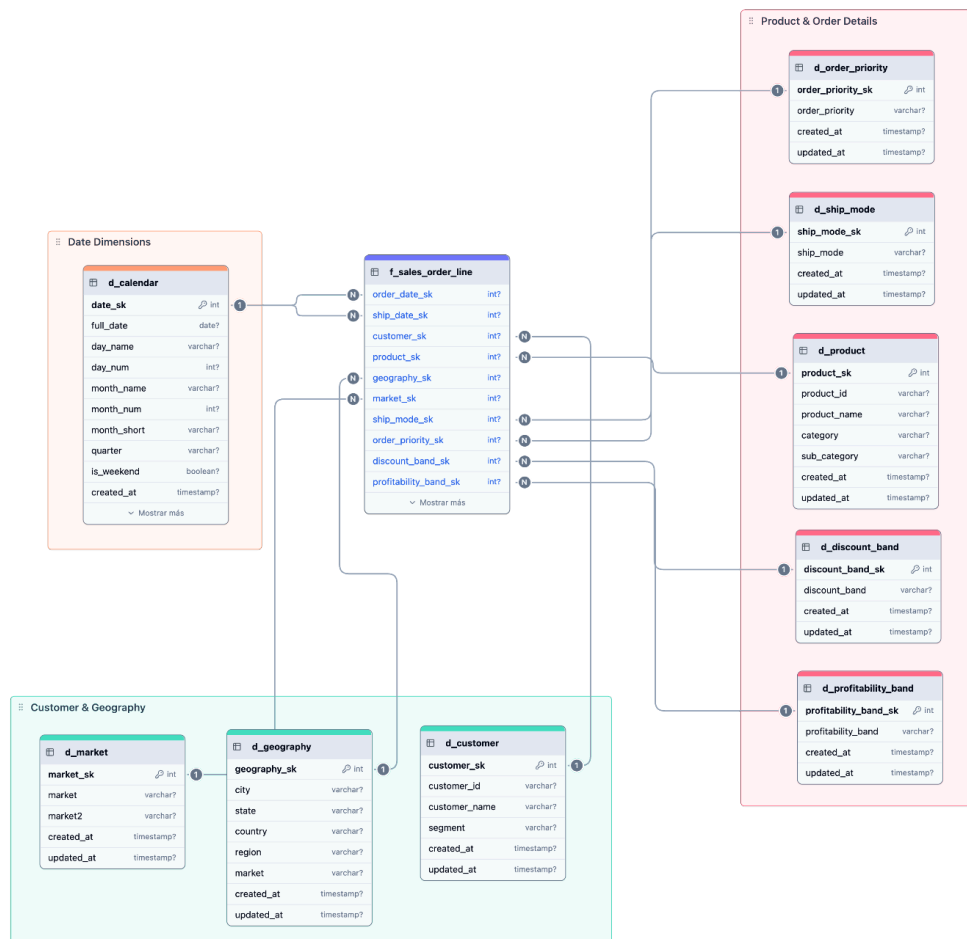


Figura 1 – Modelo dimensional final do projeto em Star Schema, desenvolvido via ChartDB.

2.2. Fatos e Dimensões

A tabela fato **f_sales_order_line** foi construída com granularidade ao nível de item de pedido, preservando o maior nível de detalhe disponível no conjunto de dados original. Cada registro representa uma transação individual associada a um produto dentro de um pedido, contendo chaves substitutas (surrogate keys) para conexão com as dimensões e medidas quantitativas necessárias à análise. Essa definição permite que todas as agregações sejam realizadas de forma consistente, sem perda de informação, suportando análises por diferentes perspectivas sem necessidade de reestruturação dos dados.

As métricas armazenadas na tabela fato incluem [sales_amount](#), [profit_amount](#), [quantity_sold](#), [discount_rate](#), [shipping_cost](#), [days_to_ship](#) e [margin_pct](#). Essas variáveis foram derivadas ou tratadas durante o processo de ETL e representam os principais indicadores necessários para avaliar o desempenho da operação sob as óticas comercial e logística. A presença simultânea dessas métricas na fato permite análises cruzadas, como impacto do desconto na margem, relação entre custo de envio e lucro, e influência do tempo de expedição sobre a rentabilidade.

As dimensões foram definidas de forma a refletir os contextos analíticos relevantes para o problema proposto.

A dimensão temporal foi estruturada como uma única tabela [d_calendar](#), construída a partir da consolidação das datas de pedido e envio. Essa abordagem permite centralizar a análise temporal em uma única dimensão, mantendo na tabela fato duas chaves distintas ([order_date_sk](#) e [ship_date_sk](#)), cada uma representando um papel específico da data no contexto analítico. Essa decisão reduz redundâncias estruturais, melhora a consistência da análise temporal e evita ambiguidades na navegação de dados na camada de visualização.

A utilização de uma única dimensão temporal associada a múltiplas chaves na tabela fato segue o padrão de role-playing dimension, permitindo diferentes interpretações temporais sobre o mesmo conjunto de dados sem duplicação estrutural. Essa abordagem melhora a consistência do modelo e reduz ambiguidades na análise.

A dimensão [d_product](#) organiza o portfólio por [product_id](#), [product_name](#), [category](#) e [sub_category](#), permitindo avaliar a contribuição de diferentes linhas de produto para o resultado financeiro. A dimensão [d_customer](#) inclui identificação e segmentação de clientes, possibilitando análises por perfil de consumo. Já a dimensão [d_geography](#) estrutura a informação territorial em níveis hierárquicos (country, region, state, city), permitindo análises espaciais detalhadas.

A dimensão [d_market](#) complementa a análise geográfica ao consolidar agrupamentos de mercado, permitindo uma visão mais agregada da operação global. No âmbito operacional, as dimensões [d_ship_mode](#) e [d_order_priority](#) permitem analisar como diferentes configurações logísticas impactam os indicadores de desempenho.

Duas dimensões adicionais foram construídas a partir de regras de transformação aplicadas durante o ETL: `d_discount_band` e `d_profitability_band`. A primeira categoriza o nível de desconto aplicado nas transações, enquanto a segunda classifica a rentabilidade com base na margem calculada. Essas dimensões têm caráter analítico e foram incluídas para facilitar segmentações e comparações, reduzindo a complexidade de interpretação de variáveis contínuas diretamente no dashboard.

A modelagem foi implementada no PostgreSQL (Neon), com criação de schema dedicado (analytics) e uso de chaves substitutas para garantir integridade e performance nos relacionamentos. As tabelas foram carregadas via scripts em Python, garantindo padronização dos tipos de dados, tratamento de nulos e consistência das chaves.

Do ponto de vista de consumo, essa estrutura impacta diretamente a camada de visualização no Power BI. A existência de uma única tabela fato conectada a dimensões bem definidas elimina ambiguidades de relacionamento, simplifica a aplicação de filtros e melhora o desempenho das consultas. Além disso, reduz a necessidade de transformações adicionais na camada de apresentação, uma vez que a lógica estrutural já foi resolvida na etapa de modelagem.

A decisão por uma arquitetura com uma única fato e dimensões normalizadas foi intencional, visando garantir estabilidade analítica, facilidade de validação e aderência às boas práticas exigidas pelo projeto. Essa abordagem assegura que os resultados apresentados sejam consistentes, rastreáveis e tecnicamente justificáveis.

Em síntese, o modelo dimensional foi projetado para suportar, de forma eficiente e estruturada, a análise da performance comercial e logística, organizando os dados em uma camada analítica coerente com os objetivos do projeto e adequada ao uso em ferramentas de Business Intelligence.

3. Integração, Tratamento e Carga de Dados

3.1. Fontes de Dados

A fonte de dados utilizada no projeto é o conjunto público [Global Superstore Dataset](#), disponibilizado na plataforma [Kaggle](#) e está acessível para download na sessão de [links](#) deste documento.

O conjunto de dados é disponibilizado em formato CSV ([superstore.csv](#)) e contém 51.290 registros e 27 colunas, com histórico entre 2011 e 2014. O conjunto de dados reúne atributos comerciais, temporais, geográficos e logísticos em uma única estrutura tabular. A granularidade está no nível de item de pedido, permitindo análises detalhadas por produto, cliente, região e operação logística.

Os principais campos disponíveis incluem:

- Identificação transacional: [row_id](#), [order_id](#)
- Datas: [order_date](#), [ship_date](#), [year](#), [weeknum](#)
- Informações do cliente: [customer_id](#), [customer_name](#), [segment](#)
- Informações do produto: [product_id](#), [product_name](#), [category](#), [sub_category](#)
- Informações geográficas: [city](#), [state](#), [region](#), [country](#)
- Informações de mercado: [market](#), [market2](#)
- Informações comerciais: [sales](#), [profit](#), [quantity](#), [discount](#)
- Informações logísticas: [ship_mode](#), [shipping_cost](#), [order_priority](#)
- Campo técnico do arquivo original: coluna renomeada no processo para [record_count](#)

Embora o conjunto já contenha informação suficiente para análise, seu formato original não é adequado para consumo analítico direto, pois não apresenta estrutura dimensional, tipagem controlada, chaves técnicas ou colunas derivadas necessárias ao modelo de BI.

Por essa razão, o dataset foi tratado como fonte bruta de entrada e reorganizado por meio de um processo de ETL, estruturado em três camadas:

- [Raw](#): dados originais (CSV)
- [Staging](#): dados tratados e padronizados
- [Curated](#): dados modelados (dimensões + fato)

A camada [raw](#) preserva o arquivo original; a camada [staging](#) contém o conjunto de dados padronizado e enriquecido; e a camada [curated](#) concentra as tabelas dimensionais e a tabela fato prontas para carga no banco PostgreSQL.

Global Superstore Dataset

This notebook is a processed version of the Global SuperStore dataset.

About Dataset

About this file
The Kaggle Global Superstore dataset is a comprehensive dataset containing information about sales and orders in a global superstore. It is a valuable resource for data analysis and visualization tasks. This dataset has been processed and transformed from its original format (txt) to CSV using the R programming language. The original dataset is available [here](#), and the transformed CSV file used in this analysis can be found [here](#).

Here is a description of the columns in the dataset:

- category: The category of products sold in the superstore.
- city: The city where the order was placed.
- country: The country in which the superstore is located.
- customer_id: A unique identifier for each customer.

Usability
10.00

License
MIT

Expected update frequency
Never

Tags
Classification
Computer Science
Regression
Marketing

Data Card Code (17) Discussion (0) Suggestions (0)

Detail Compact Column 10 of 27 columns

Here is a description of the columns in the dataset:

- category: The category of products sold in the superstore.
- city: The city where the order was placed.
- country: The country in which the superstore is located.
- customer_id: A unique identifier for each customer.
- customer_name: The name of the customer who placed the order.
- discount: The discount applied to the order.
- market: The market or region where the superstore operates.
- ji_lu_shu: An unknown or unspecified column.
- order_date: The date when the order was placed.

Category	City	Country	Customer.ID	Customer.Name	Discount
The category of products sold in the superstore	The city where the order was placed	The country in which the superstore is located	A unique identifier for each customer	The name of the customer who placed the order	The discount applied to the order
Office Supplies 61%	New York City 2%		4873 unique values	795 unique values	
Technology 20%	Los Angeles 1%				
Other (9876) 19%	Other (49628) 97%				

Summary

- 1 file
- .csv 1
- 27 columns
- String 15
- Integer 6
- DateTime 2
- Other 4

Figura 2 e 3 – Página de download do dataset Global Superstore, no Kaggle.

3.2 Processos de Integração e Carga (ETL)

O processo de integração, tratamento e carga foi desenvolvido integralmente em Python, com execução em ambiente Windows no Parallels. Foram utilizadas principalmente as bibliotecas:

- **pandas** (manipulação de dados)
- **sqlalchemy** (conexão com banco)
- **psycopg2-binary** (driver PostgreSQL)
- **python-dotenv** (gestão de credenciais)

A decisão de centralizar o ETL em Python foi adotada para garantir maior controle sobre o processo de transformação, permitir automação completa e evitar dependência de lógica distribuída em múltiplas ferramentas. Em vez de realizar transformações diretamente no Power BI, optou-se por construir uma camada analítica preparada previamente, permitindo maior controle sobre tipos de dados, padronização de colunas, criação de chaves, implementação de regras derivadas e validação da consistência dos resultados. Essa abordagem também reduz a carga de transformação na camada visual e melhora o desempenho do modelo final.

Do ponto de vista operacional, o pipeline foi estruturado em três etapas principais e independentes, com o intuito de separar claramente engenharia de dados local, persistência em nuvem e validação de consistência. Essa divisão seguiu a seguinte regra:

- [01_etl_base.py](#): leitura, tratamento, modelagem local e exportação da camada curada
- [02_load_to_neon.py](#): carga das tabelas da camada curada para o PostgreSQL no Neon
- [03_validate_neon.py](#): validação analítica da tabela fato diretamente no banco

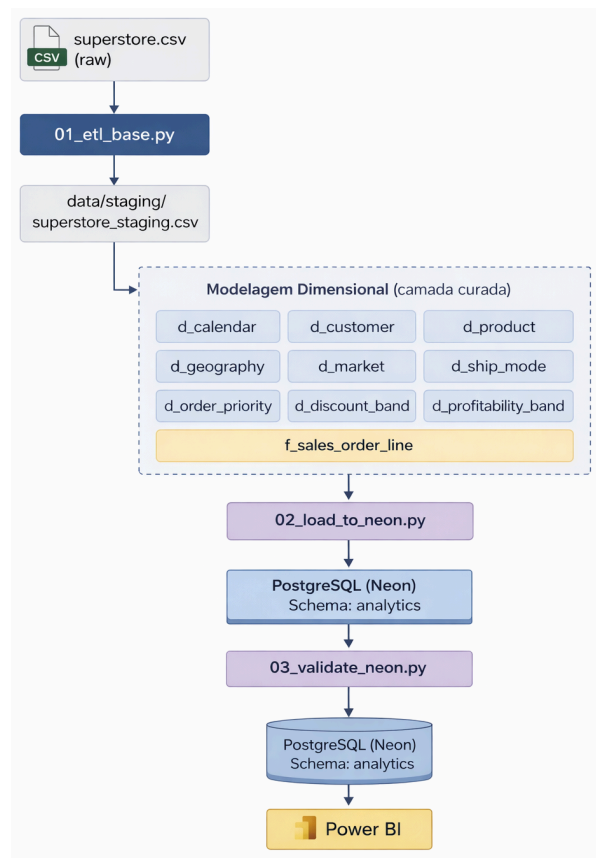


Figura 4 – Fluxo de integração, tratamento, modelagem e carga da base analítica.

3.2.1 Preparação do ambiente e estrutura local

Antes da execução dos scripts, foi estruturado um diretório local contendo as pastas necessárias ao pipeline. Essa organização teve como objetivo separar o conjunto de dados bruto, os artefatos intermediários, os arquivos curados, os scripts e os registros de validação. A estrutura principal adotada foi composta por:

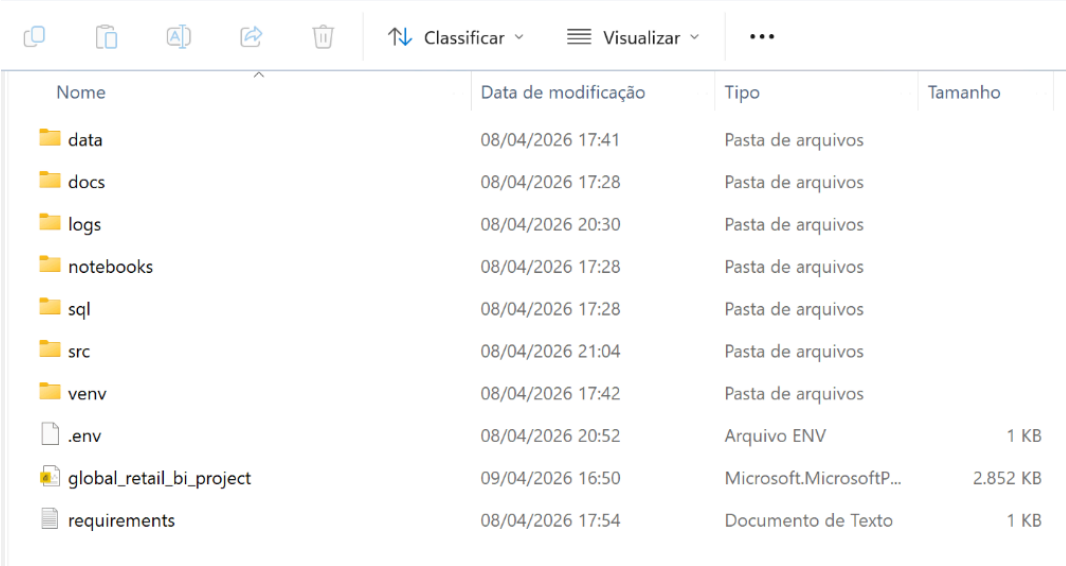
```
tcc_analytics_bi/  
├── data/  
│   ├── raw/  
│   ├── staging/  
│   └── curated/  
├── src/  
├── sql/  
├── logs/  
└── .env
```

A separação entre raw, staging e curated foi mantida ao longo de todo o projeto porque permite distinguir claramente o que é dado original, o que é dado tratado e o que é dado já modelado para consumo analítico. Isso facilita auditoria, reprocessamento e documentação técnica.

Ou seja, ao manter o arquivo bruto isolado, evita-se que a fonte seja sobrescrita durante as transformações. Ao salvar a camada staging, torna-se possível revisar o conjunto de dados já tratado antes da modelagem final. Ao separar a camada curated, garante-se que apenas as tabelas já estruturadas sejam carregadas no banco.

Comandos executados no Prompt de Comando (cmd) para estruturar o projeto:

```
<\> cmd  
  
cd C:\Users\paulafreire\Documents\tcc_analytics_bi  
mkdir data  
mkdir data\raw  
mkdir data\staging  
mkdir data\curated  
mkdir src  
mkdir sql  
mkdir sql\ddl  
mkdir sql\validation  
mkdir logs
```



Nome	Data de modificação	Tipo	Tamanho
data	08/04/2026 17:41	Pasta de arquivos	
docs	08/04/2026 17:28	Pasta de arquivos	
logs	08/04/2026 20:30	Pasta de arquivos	
notebooks	08/04/2026 17:28	Pasta de arquivos	
sql	08/04/2026 17:28	Pasta de arquivos	
src	08/04/2026 21:04	Pasta de arquivos	
venv	08/04/2026 17:42	Pasta de arquivos	
.env	08/04/2026 20:52	Arquivo ENV	1 KB
global_retail_bi_project	09/04/2026 16:50	Microsoft.MicrosoftP...	2.852 KB
requirements	08/04/2026 17:54	Documento de Texto	1 KB

Figura 5 – Diretório Windows organizado para o projeto.

A primeira etapa do pipeline consistiu na leitura do arquivo `superstore.csv` a partir da camada `raw`. Essa leitura foi feita com atenção ao encoding, pois o conjunto de dados continha um campo com nome corrompido por caracteres especiais. O objetivo do profiling inicial foi identificar a estrutura do conjunto de dados antes de qualquer transformação: quantidade de linhas, quantidade de colunas, nomes originais dos campos, tipos inferidos automaticamente e possíveis anomalias estruturais.

</> Python

```
import pandas as pd

file_path =
r"C:\Users\paulafreire\Documents\tcc_analytics_bi\data\raw\superstore.csv"

df = pd.read_csv(file_path, encoding="latin1")

print(df.head())
print(df.info())
print(df.columns.tolist())
```

A leitura confirmou a existência de 51.290 linhas e 27 colunas. Também mostrou que os campos de data estavam sendo interpretados inicialmente como texto, que havia mistura de inteiros e decimais nas métricas e que uma das colunas possuía nome corrompido. Esse diagnóstico justificou tecnicamente a etapa seguinte de normalização estrutural.

3.2.2 Leitura do arquivo bruto e inspeção inicial

A primeira etapa do script `01_etl_base.py` consiste na leitura do arquivo `superstore.csv` diretamente da camada `raw`. O caminho do projeto foi definido por meio de `Path`, permitindo organizar a navegação entre pastas de maneira padronizada e explícita. O arquivo foi lido com `encoding="latin1"`, pois o conjunto de dados apresentava um campo com problema de encoding que exigia tratamento posterior.

`</> Python`

```
BASE_DIR = Path(r"C:\Users\paulafreire\Documents\tcc_analytics_bi")
RAW_FILE = BASE_DIR / "data" / "raw" / "superstore.csv"

df = pd.read_csv(RAW_FILE, encoding="latin1")
```

```
# =====
# CONFIGURAÇÃO DE PASTAS
# =====
BASE_DIR = Path(r"C:\Users\paulafreire\Documents\tcc_analytics_bi")
RAW_FILE = BASE_DIR / "data" / "raw" / "superstore.csv"
STAGING_DIR = BASE_DIR / "data" / "staging"
CURATED_DIR = BASE_DIR / "data" / "curated"
LOGS_DIR = BASE_DIR / "logs"

STAGING_DIR.mkdir(parents=True, exist_ok=True)
CURATED_DIR.mkdir(parents=True, exist_ok=True)
LOGS_DIR.mkdir(parents=True, exist_ok=True)
```

Nessa fase, o objetivo não era ainda transformar os dados, mas confirmar que a leitura ocorreu corretamente, identificar a quantidade de linhas e colunas e registrar o estado bruto da fonte antes de qualquer modificação. Essa etapa é importante porque estabelece a linha de base do processo: a partir dela, torna-se possível demonstrar que a granularidade original foi preservada até a construção da tabela fato.

3.2.3 Padronização dos nomes das colunas

Após a leitura inicial, foi realizada a padronização dos nomes das colunas. Essa etapa foi necessária porque o conjunto de dados original possuía nomes com espaços, pontos e pelo menos um campo com caracteres corrompidos, sendo necessário transformar todos os nomes para minúsculas, substituir espaços e pontos por underscore, remover caracteres especiais e corrigir manualmente a

coluna afetada por encoding. Esses elementos dificultam o uso consistente em Python, SQL e Power BI. Para resolver isso, foi criada a função `normalize_column_name`.

```
def normalize_column_name(col: str) -> str:
    col = str(col).strip().lower()
    col = col.replace(".", "_").replace(" ", "_").replace("-", "_").replace("/", "_")
    col = unicodedata.normalize("NFKD", col).encode("ascii", "ignore").decode("utf-8")
    col = re.sub(r"^[a-z0-9_]", "", col)
    col = re.sub(r"_+", "_", col).strip("_")
    return col
```

Essa decisão foi importante por três motivos:

- Reduzir a possibilidade de erro em scripts Python e queries SQL;
- Facilitar a manutenção do código, pois eliminam ambiguidades de escrita;
- Garantir compatibilidade com o PostgreSQL e com o Power BI, evitando falhas causadas por caracteres acentuados, espaços ou símbolos não reconhecidos.

Depois da normalização geral, o script compara os nomes resultantes com um conjunto de colunas esperadas. Caso alguma coluna não corresponda ao padrão conhecido, ela é identificada como “coluna não mapeada”. Quando há apenas uma coluna fora do padrão (como ocorreu neste projeto) ela é renomeada para `record_count`. Essa decisão preserva a coluna no dataset tratado, sem permitir que um nome corrompido comprometa o restante do pipeline.


```

# =====
# PADRONIZAÇÃO DOS NOMES DE COLUNAS
# =====
df.columns = [normalize_column_name(c) for c in df.columns]

expected_columns = {
    "category",
    "city",
    "country",
    "customer_id",
    "customer_name",
    "discount",
    "market",
    "order_date",
    "order_id",
    "order_priority",
    "product_id",
    "product_name",
    "profit",
    "quantity",
    "region",
    "row_id",
    "sales",
    "segment",
    "ship_date",
    "ship_mode",
    "shipping_cost",
    "state",
    "sub_category",
    "year",
    "market2",
    "weeknum",
}

weird_cols = [c for c in df.columns if c not in expected_columns]
if weird_cols:
    print(f"Colunas não mapeadas detectadas: {weird_cols}")

if len(weird_cols) == 1:
    df = df.rename(columns={weird_cols[0]: "record_count"})
elif len(weird_cols) > 1:
    rename_map = {c: f"extra_col_{i+1}" for i, c in enumerate(weird_cols)}
    df = df.rename(columns=rename_map)

print("\nColunas após padronização:")
print(df.columns.tolist())

```

Essa padronização é importante porque unifica a camada analítica e reduz o risco de inconsistência em scripts futuros, joins e consultas.

3.2.4 Tipagem dos campos e tratamento básico de qualidade

Com a estrutura já padronizada, foi realizada a tipagem explícita dos campos. O script separa colunas textuais, numéricas inteiras, numéricas decimais e datas. Campos de data, por exemplo, precisam ser convertidos corretamente para permitir criação de chaves temporais; métricas monetárias precisam ter tratamento decimal;

e identificadores inteiros devem estar consistentes para validações e controle de granularidade.

Desta forma, as colunas textuais foram convertidas para string e higienizadas com remoção de espaços excedentes; as colunas numéricas foram convertidas para os tipos apropriados, diferenciando-se inteiros e decimais e os campos `order_date` e `ship_date` foram convertidos para datetime, permitindo a derivação posterior das chaves temporais e do indicador de prazo entre pedido e envio.

```
# =====
# TRATAMENTO DE TIPOS
# =====
print("\nTratando tipos...")

text_cols = [
    "category",
    "city",
    "country",
    "customer_id",
    "customer_name",
    "market",
    "order_id",
    "order_priority",
    "product_id",
    "product_name",
    "region",
    "segment",
    "ship_mode",
    "state",
    "sub_category",
    "market2",
]

for col in text_cols:
    if col in df.columns:
        df[col] = df[col].astype(str).str.strip()

numeric_cols_int = ["quantity", "row_id", "year", "weeknum"]
for col in numeric_cols_int:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce").astype("Int64")

numeric_cols_decimal = ["discount", "profit", "sales", "shipping_cost"]
for col in numeric_cols_decimal:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

for col in ["order_date", "ship_date"]:
    if col in df.columns:
        df[col] = pd.to_datetime(df[col], errors="coerce")
```

Na sequência, foi aplicado um tratamento básico de qualidade. A primeira regra verifica duplicidade em `row_id`, uma vez que essa coluna representa a chave

de negócio do nível transacional. Depois, são eliminados registros com ausência de campos críticos, como `row_id`, `order_id`, `order_date`, `ship_date`, `customer_id` e `product_id`. Em seguida, campos textuais vazios são preenchidos com "Unknown" e métricas numéricas nulas são preenchidas com zero. O objetivo dessa etapa foi garantir consistência mínima do conjunto de dados antes da modelagem, sem deslocar para o Power BI o tratamento de problemas que já poderiam ser resolvidos no ETL.

```
# =====
# TRATAMENTO BÁSICO DE QUALIDADE
# =====
print("\nAplicando regras de qualidade...\n")

if "row_id" in df.columns:
    before = len(df)
    df = df.drop_duplicates(subset=["row_id"]).copy()
    after = len(df)
    print(f"Duplicidades removidas por row_id: {before - after}")

required_cols = ["row_id", "order_id", "order_date", "ship_date", "customer_id", "product_id"]
before = len(df)
df = df.dropna(subset=[c for c in required_cols if c in df.columns]).copy()
after = len(df)
print(f"Linhas removidas por falta de campos críticos: {before - after}")

for col in text_cols:
    if col in df.columns:
        df[col] = df[col].replace("", pd.NA).fillna("Unknown")

for col in ["discount", "profit", "sales", "shipping_cost"]:
    if col in df.columns:
        df[col] = df[col].fillna(0)

if "quantity" in df.columns:
    df["quantity"] = df["quantity"].fillna(0).astype(int)

if "row_id" in df.columns:
    df["row_id"] = df["row_id"].astype(int)

print(f"Linhas após limpeza: {len(df):,}")
```

3.2.5 Criação de variáveis derivadas e geração da camada staging

Após a limpeza estrutural, o script gera variáveis derivadas a serem utilizadas tanto no modelo dimensional quanto nas análises de negócio. Foram criadas as chaves temporais `order_date_sk` e `ship_date_sk`, calculadas no formato `yyyymmdd`, permitindo a integração com a dimensão calendário. Também foi calculado o indicador `days_to_ship`, correspondente ao número de dias entre pedido e envio.

No âmbito das métricas, foram definidos os campos monetários padronizados `sales_amount`, `profit_amount` e `shipping_cost_amount`; o percentual de margem `margin_pct`; o indicador `order_line_count`, fixado em 1 para facilitar contagens; e o

campo booleano `profitable_flag`, que classifica se a linha gerou lucro positivo. Também foram geradas duas classificações: `discount_band` e `profitability_band`, que permitem segmentar o comportamento de desconto e rentabilidade em faixas analíticas, facilitando a interpretação e a exploração dos dados na camada de visualização.

```
# =====
# COLUNAS DERIVADAS
# =====
print("\nGerando colunas derivadas...")

df["order_date_sk"] = df["order_date"].dt.strftime("%Y%m%d").astype(int)
df["ship_date_sk"] = df["ship_date"].dt.strftime("%Y%m%d").astype(int)
df["days_to_ship"] = (df["ship_date"] - df["order_date"]).dt.days
df["days_to_ship"] = df["days_to_ship"].fillna(0).astype(int)

df["sales_amount"] = df["sales"].round(2)
df["profit_amount"] = df["profit"].round(2)
df["quantity_sold"] = df["quantity"].astype(int)
df["discount_rate"] = df["discount"].round(4)
df["shipping_cost_amount"] = df["shipping_cost"].round(2)

df["margin_pct"] = df.apply(
    lambda x: round(x["profit_amount"] / x["sales_amount"], 4) if x["sales_amount"] not in
    axis=1,
)

df["order_line_count"] = 1
df["profitable_flag"] = df["profit_amount"] > 0

df["discount_band"] = df["discount_rate"].apply(discount_band)
df["profitability_band"] = df["margin_pct"].apply(profitability_band)
```

Essas variáveis foram materializadas no ETL porque fazem parte da lógica analítica central do projeto. Ao criá-las antes da carga no banco, garante-se que os mesmos conceitos sejam usados em todas as etapas seguintes, sem necessidade de reconstrução por medidas ou colunas calculadas no Power BI.

Após essa etapa, o conjunto de dados já tratado foi salvo como `superstore_staging.csv` na pasta `data/staging`.

3.2.6 Construção das dimensões

Concluída a camada de staging, iniciou-se a construção da camada `curated`. Nessa etapa, foram geradas as dimensões do modelo por meio da seleção, ordenação, deduplicação e criação de chaves substitutas. A lógica foi implementada diretamente no script por tabela, utilizando uma função auxiliar para gerar as surrogate keys em sequência numérica. Essa escolha foi adotada porque simplifica a modelagem local e garante reprodutibilidade completa do processo.

```
def add_surrogate_key(df: pd.DataFrame, sk_name: str) -> pd.DataFrame:
    df = df.reset_index(drop=True).copy()
    df[sk_name] = range(1, len(df) + 1)
    cols = [sk_name] + [c for c in df.columns if c != sk_name]
    return df[cols]
```

As dimensões `d_customer`, `d_product`, `d_geography`, `d_market`, `d_ship_mode`, `d_order_priority`, `d_discount_band` e `d_profitability_band` foram construídas a partir da seleção dos atributos relevantes e remoção de duplicidades. A dimensão temporal foi construída como uma tabela única (`d_calendar`), gerada a partir da consolidação das datas de pedido e envio presentes no conjunto de dados. Essa abordagem garante consistência analítica e evita duplicidade estrutural, permitindo que diferentes contextos temporais sejam analisados a partir de uma única dimensão.

```
# =====
# CONSTRUÇÃO DAS DIMENSÕES
# =====
print("\nConstruindo dimensões...")

d_calendar = build_calendar_dimension(df["order_date"], df["ship_date"])

d_customer = (
    df[["customer_id", "customer_name", "segment"]]
    .sort_values(["customer_id", "customer_name", "segment"])
    .drop_duplicates(subset=["customer_id"], keep="first")
    .reset_index(drop=True)
)
d_customer = add_surrogate_key(d_customer, "customer_sk")

d_product = (
    df[["product_id", "product_name", "category", "sub_category"]]
    .sort_values(["product_id", "product_name", "category", "sub_category"])
    .drop_duplicates(subset=["product_id"], keep="first")
    .reset_index(drop=True)
)
d_product = add_surrogate_key(d_product, "product_sk")

d_geography = (
    df[["country", "region", "state", "city"]]
    .drop_duplicates()
    .sort_values(["country", "region", "state", "city"])
    .reset_index(drop=True)
)
d_geography = add_surrogate_key(d_geography, "geography_sk")

d_market = (
    df[["market", "market2"]]
    .sort_values(["market", "market2"])
    .drop_duplicates(subset=["market"], keep="first")
    .reset_index(drop=True)
)
d_market = add_surrogate_key(d_market, "market_sk")
```

```

d_ship_mode = (
    df[["ship_mode"]]
    .drop_duplicates()
    .sort_values(["ship_mode"])
    .reset_index(drop=True)
)
d_ship_mode = add_surrogate_key(d_ship_mode, "ship_mode_sk")

d_order_priority = (
    df[["order_priority"]]
    .drop_duplicates()
    .sort_values(["order_priority"])
    .reset_index(drop=True)
)
d_order_priority = add_surrogate_key(d_order_priority, "order_priority_sk")

d_discount_band = (
    df[["discount_band"]]
    .drop_duplicates()
    .sort_values(["discount_band"])
    .reset_index(drop=True)
)
d_discount_band = add_surrogate_key(d_discount_band, "discount_band_sk")

d_profitability_band = (
    df[["profitability_band"]]
    .drop_duplicates()
    .sort_values(["profitability_band"])
    .reset_index(drop=True)
)
d_profitability_band = add_surrogate_key(d_profitability_band, "profitability_band_sk")

```

```

def build_calendar_dimension(order_dates: pd.Series, ship_dates: pd.Series) -> pd.DataFrame:
    all_dates = pd.concat([order_dates, ship_dates], ignore_index=True)
    all_dates = pd.to_datetime(all_dates, errors="coerce").dropna().drop_duplicates().sort_values()

    d = pd.DataFrame({"full_date": all_dates})
    d["date_sk"] = d["full_date"].dt.strftime("%Y%m%d").astype(int)
    d["year"] = d["full_date"].dt.year
    d["quarter"] = d["full_date"].dt.quarter
    d["month_num"] = d["full_date"].dt.month
    d["month_name"] = d["full_date"].dt.month_name()
    d["month_short"] = d["full_date"].dt.strftime("%b")
    d["year_month"] = d["full_date"].dt.strftime("%Y-%m")
    d["year_month_sort"] = d["full_date"].dt.year * 100 + d["full_date"].dt.month
    d["week_num"] = d["full_date"].dt.isocalendar().week.astype(int)
    d["day_num"] = d["full_date"].dt.day
    d["day_name"] = d["full_date"].dt.day_name()
    d["is_weekend"] = d["full_date"].dt.dayofweek.isin([5, 6])

    return d[
        [
            "date_sk",
            "full_date",
            "year",
            "quarter",
            "month_num",
            "month_name",
            "month_short",
            "year_month",
            "year_month_sort",
            "week_num",
            "day_num",
            "day_name",
            "is_weekend",
        ]
    ]

```

Essa etapa foi importante porque transformou o conjunto de dados originalmente tabular em componentes estruturados do modelo dimensional. Cada

dimensão passou a representar um eixo de análise específico, enquanto as chaves substitutas prepararam o caminho para a montagem da fato.

3.2.7 Construção da tabela fato e validações críticas

A tabela fato foi construída a partir da cópia do conjunto de dados tratado e da junção com cada uma das dimensões, recuperando as respectivas surrogate keys. Cada merge foi executado com `validate="many_to_one"`, o que significa que múltiplas linhas do conjunto de dados transacional podem apontar para um único registro da dimensão correspondente. Essa validação foi adotada para garantir que a granularidade original do conjunto de dados fosse preservada ao longo das junções.

```
# =====
# CONSTRUÇÃO DA FATO
# =====
print("Construindo tabela fato...")

fact = df.copy()

fact = fact.merge(
    d_customer[["customer_sk", "customer_id"]],
    on="customer_id",
    how="left",
    validate="many_to_one"
)

fact = fact.merge(
    d_product[["product_sk", "product_id"]],
    on="product_id",
    how="left",
    validate="many_to_one"
)

fact = fact.merge(
    d_geography[["geography_sk", "country", "region", "state", "city"]],
    on=["country", "region", "state", "city"],
    how="left",
    validate="many_to_one"
)

fact = fact.merge(
    d_market[["market_sk", "market"]],
    on="market",
    how="left",
    validate="many_to_one"
)

fact = fact.merge(
    d_ship_mode[["ship_mode_sk", "ship_mode"]],
    on="ship_mode",
    how="left",
    validate="many_to_one"
)
```

```

fact = fact.merge(
    d_order_priority[["order_priority_sk", "order_priority"]],
    on="order_priority",
    how="left",
    validate="many_to_one"
)

fact = fact.merge(
    d_discount_band[["discount_band_sk", "discount_band"]],
    on="discount_band",
    how="left",
    validate="many_to_one"
)

fact = fact.merge(
    d_profitability_band[["profitability_band_sk", "profitability_band"]],
    on="profitability_band",
    how="left",
    validate="many_to_one"
)

```

Depois de associar todas as chaves dimensionais, a tabela `f_sales_order_line` foi materializada com as colunas analíticas finais e renomeação dos campos de negócio:

```

f_sales_order_line = fact[
    [
        "row_id",
        "order_id",
        "order_date_sk",
        "ship_date_sk",
        "customer_sk",
        "product_sk",
        "geography_sk",
        "market_sk",
        "ship_mode_sk",
        "order_priority_sk",
        "discount_band_sk",
        "profitability_band_sk",
        "sales_amount",
        "profit_amount",
        "quantity_sold",
        "discount_rate",
        "shipping_cost_amount",
        "days_to_ship",
        "margin_pct",
        "order_line_count",
        "profitable_flag",
    ]
].copy()

f_sales_order_line = f_sales_order_line.rename(
    columns={
        "row_id": "row_id_bk",
        "order_id": "order_id_bk",
        "shipping_cost_amount": "shipping_cost",
    }
)

f_sales_order_line = add_surrogate_key(f_sales_order_line, "sales_line_sk")

```

Ao final, a fato recebeu também a surrogate key `sales_line_sk`.

A seguir, o script executa as validações críticas do processo: verificação de nulos em todas as FKs, comparação entre a quantidade de linhas do conjunto de dados limpo e da fato, e checagem de duplicidade de `row_id_bk`. Essas validações são fundamentais porque demonstram que a modelagem analítica preservou corretamente a granularidade da origem e que nenhuma linha foi indevidamente multiplicada ou perdida.

```
# =====
# VALIDAÇÕES CRÍTICAS
# =====
print("\nValidando chaves nulas nas dimensões e fato...")

fk_cols = [
    "order_date_sk",
    "ship_date_sk",
    "customer_sk",
    "product_sk",
    "geography_sk",
    "market_sk",
    "ship_mode_sk",
    "order_priority_sk",
    "discount_band_sk",
    "profitability_band_sk",
]

null_fk_report = {col: int(f_sales_order_line[col].isna().sum()) for col in fk_cols}
print(null_fk_report)

print("\nValidando granularidade da fato...")
print(f"Linhas raw limpas: {len(df):,}")
print(f"Linhas fato: {len(f_sales_order_line):,}")

if len(df) != len(f_sales_order_line):
    raise ValueError(
        f"ERRO: a fato ficou com {len(f_sales_order_line):,} linhas, "
        f"mas a base limpa tem {len(df):,}. Houve multiplicação indevida."
    )

duplicated_row_id = int(f_sales_order_line["row_id_bk"].duplicated().sum())
print(f"Duplicidades de row_id_bk na fato: {duplicated_row_id}")

if duplicated_row_id > 0:
    raise ValueError(
        f"ERRO: existem {duplicated_row_id} row_id_bk duplicados na fato."
    )
```

O resultado final dessa etapa confirmou 51.290 linhas tanto no conjunto de dados limpo quanto na fato, sem duplicidade de `row_id_bk` e sem chaves estrangeiras nulas.

```

Prompt de Comando
{'order_date_sk': 0, 'ship_date_sk': 0, 'customer_sk': 0, 'product_sk': 0, 'geography_sk': 0, 'market_sk': 0, 'ship_mode_sk': 0, 'order_priority_sk': 0, 'discount_band_sk': 0, 'profitability_band_sk': 0}

Validando granularidade da fato...
Linhas raw limpas: 51,290
Linhas fato: 51,290
Duplicidades de row_id_bk na fato: 0

Salvando arquivos curated...

===== RESUMO FINAL =====
Staging: C:\Users\paulafreire\Documents\tcc_analytics_bi\data\staging\superstore_staging.csv
Curated: C:\Users\paulafreire\Documents\tcc_analytics_bi\data\curated
Logs: C:\Users\paulafreire\Documents\tcc_analytics_bi\logs

Quantidade de registros por tabela:
table_name  row_count
d_calendar  1468
d_customer  4873
d_product   10292
d_geography  3819
d_market     7
d_ship_mode  4
d_order_priority  4
d_discount_band  5
d_profitability_band  5
f_sales_order_line  51290

Primeiras linhas da fato:
sales_line_sk  row_id_bk  order_id_bk  order_date_sk  ...  days_to_ship  margin_pct  order_line_count  profitable_flag
0              1      36624      CA-2011-130813  20110107  ...           2           0.4911           1             True
1              2      37033      CA-2011-148614  20110121  ...           5           0.4889           1             True
2              3      31468      CA-2011-118962  20110805  ...           4           0.4686           1             True
3              4      31469      CA-2011-118962  20110805  ...           4           0.4798           1             True
4              5      32440      CA-2011-146969  20110929  ...           4           0.5183           1             True

[5 rows x 22 columns]

Processo ETL local concluído com sucesso.

```

3.2.8 Exportação da camada curada e geração do relatório de qualidade

Após a construção das dimensões e da fato, todas as tabelas foram exportadas em CSV para a pasta [data/curated](#). Essa etapa foi importante porque criou um conjunto de artefatos intermediários independentes do banco, permitindo tanto inspeção manual quanto carga posterior no PostgreSQL. Além disso, o script gerou um relatório de qualidade em formato CSV ([quality_report.csv](#)), contendo a quantidade de linhas de cada tabela analítica.

```

# =====
# EXPORTAR CURATED
# =====
print("\nSalvando arquivos curated..")

d_calendar.to_csv(CURATED_DIR / "d_calendar.csv", index=False, encoding="utf-8-sig")
d_customer.to_csv(CURATED_DIR / "d_customer.csv", index=False, encoding="utf-8-sig")
d_product.to_csv(CURATED_DIR / "d_product.csv", index=False, encoding="utf-8-sig")
d_geography.to_csv(CURATED_DIR / "d_geography.csv", index=False, encoding="utf-8-sig")
d_market.to_csv(CURATED_DIR / "d_market.csv", index=False, encoding="utf-8-sig")
d_ship_mode.to_csv(CURATED_DIR / "d_ship_mode.csv", index=False, encoding="utf-8-sig")
d_order_priority.to_csv(CURATED_DIR / "d_order_priority.csv", index=False, encoding="utf-8-sig")
d_discount_band.to_csv(CURATED_DIR / "d_discount_band.csv", index=False, encoding="utf-8-sig")
d_profitability_band.to_csv(CURATED_DIR / "d_profitability_band.csv", index=False, encoding="utf-8-sig")
f_sales_order_line.to_csv(CURATED_DIR / "f_sales_order_line.csv", index=False, encoding="utf-8-sig")

```

3.2.9 Carga no PostgreSQL (Neon)

Com a camada [curated](#) concluída, a etapa seguinte foi a carga no PostgreSQL hospedado na plataforma Neon. Essa etapa foi executada no script [02_load_to_neon.py](#), que lê os arquivos CSV da camada curated, garante a existência do schema [analytics](#) e carrega as tabelas em ordem lógica: primeiro as dimensões, depois a fato.

O uso do Neon foi definido para disponibilizar a base analítica em ambiente cloud, separado do ambiente local de ETL. Isso facilita consumo no Power BI, validação externa e organização da solução em camadas distintas.

```
import os
from pathlib import Path

import pandas as pd
from sqlalchemy import create_engine, text
from dotenv import load_dotenv

load_dotenv()

BASE_DIR = Path(r"C:\Users\paulafreire\Documents\tcc_analytics_bi")
CURATED_DIR = BASE_DIR / "data" / "curated"

conn_str = os.getenv("NEON_CONNECTION_STRING")
engine = create_engine(conn_str)

tables_in_order = [
    "d_calendar",
    "d_customer",
    "d_product",
    "d_geography",
    "d_market",
    "d_ship_mode",
    "d_order_priority",
    "d_discount_band",
    "d_profitability_band",
    "f_sales_order_line",
]
```

```
print("Garantindo schema analytics...")
with engine.connect() as conn:
    conn.execute(text("CREATE SCHEMA IF NOT EXISTS analytics"))
    conn.commit()

for table_name in tables_in_order:
    file_path = CURATED_DIR / f"{table_name}.csv"

    print(f"\nLendo {table_name}...")
    df = pd.read_csv(file_path)
    print(f"Linhas: {len(df)}")

    print(f"Carregando analytics.{table_name} ...")
    df.to_sql(
        name=table_name,
        con=engine,
        schema="analytics",
        if_exists="replace",
        index=False,
        method="multi",
        chunksize=1000
    )

    print(f"analytics.{table_name} carregada com sucesso.")

print("\nCarga completa finalizada com sucesso.")
```

```

Prompt de Comando - pythor  X  +  v

C:\Users\paulafreire\Documents\tcc_analytics_bi>python src\02_load_to_neon.py
Garantindo schema analytics...

Lendo d_calendar...
Linhas: 1468
Carregando analytics.d_calendar ...
analytics.d_calendar carregada com sucesso.

Lendo d_customer...
Linhas: 4873
Carregando analytics.d_customer ...
analytics.d_customer carregada com sucesso.

Lendo d_product...
Linhas: 10292
Carregando analytics.d_product ...
analytics.d_product carregada com sucesso.

Lendo d_geography...
Linhas: 3819
Carregando analytics.d_geography ...
analytics.d_geography carregada com sucesso.

Lendo d_market...
Linhas: 7
Carregando analytics.d_market ...
analytics.d_market carregada com sucesso.

Lendo d_ship_mode...
Linhas: 4
Carregando analytics.d_ship_mode ...
analytics.d_ship_mode carregada com sucesso.

Lendo d_order_priority...
Linhas: 4
Carregando analytics.d_order_priority ...
analytics.d_order_priority carregada com sucesso.

```

Prompt utilizado para criar o arquivo de ambiente:

```
<\> cmd
```

```
notepad .env
```

Formato do arquivo `.env` utilizado no projeto:

```
<\> env
```

```
NEON_CONNECTION_STRING=postgresql+psycopg2://USUARIO:SENHA@HOST
/neondb?sslmode=require&channel_binding=require
```

3.2.10 Validação analítica no banco

Após a carga, foram executadas validações diretamente no [PostgreSQL](#) por meio do script [03_validate_neon.py](#). Diferentemente do relatório de qualidade local, essa etapa confirma que os dados persistidos em nuvem mantiveram a consistência esperada na camada analítica final. O script atual não executa mais apenas contagem estrutural de todas as tabelas; ele valida diretamente as principais métricas da fato: total de linhas, vendas totais, lucro total, quantidade total e número de pedidos distintos.

```

import os
from sqlalchemy import create_engine, text
from dotenv import load_dotenv

load_dotenv()

conn_str = os.getenv("NEON_CONNECTION_STRING")
engine = create_engine(conn_str)

queries = {
    "total_linhas_fato": """
        SELECT COUNT(*)
        FROM analytics.f_sales_order_line
    """,
    "total_vendas": """
        SELECT ROUND(SUM(sales_amount)::numeric, 2)
        FROM analytics.f_sales_order_line
    """,
    "total_lucro": """
        SELECT ROUND(SUM(profit_amount)::numeric, 2)
        FROM analytics.f_sales_order_line
    """,
    "total_quantidade": """
        SELECT SUM(quantity_sold)
        FROM analytics.f_sales_order_line
    """,
    "total_pedidos_distintos": """
        SELECT COUNT(DISTINCT order_id_bk)
        FROM analytics.f_sales_order_line
    """,
}

with engine.connect() as conn:
    print("VALIDAÇÃO ANALÍTICA DA FATO\n")
    for nome, query in queries.items():
        result = conn.execute(text(query)).scalar()
        print(f"{nome}: {result}")

```

Os resultados confirmados no projeto foram:

- **total_linhas_fato:** 51.290
- **total_vendas:** 12.642.905,00
- **total_lucro:** 1.467.456,52
- **total_quantidade:** 178.312
- **total_pedidos_distintos:** 25.035

```

C:\Users\paulafreire\Documents\tcc_analytics_bi>python src\03_validate_neon.py
VALIDAÇÃO ANALÍTICA DA FATO

total_linhas_fato: 51290
total_vendas: 12642905.00
total_lucro: 1467456.52
total_quantidade: 178312
total_pedidos_distintos: 25035

C:\Users\paulafreire\Documents\tcc_analytics_bi>

```

Esses totais são relevantes porque posteriormente são usados como base de homologação com o Power BI.

3.2.11 Consolidação técnica do processo

Em termos técnicos, o pipeline foi desenhado para garantir reprodutibilidade, porque pode ser executado novamente a partir do arquivo bruto; rastreabilidade, porque cada camada gera artefatos próprios; consistência, porque as regras analíticas foram materializadas antes da visualização; e governança, porque as validações foram incorporadas ao próprio processo. Em síntese, a etapa de integração, tratamento e carga constituiu a construção da base analítica do projeto, convertendo um arquivo transacional bruto em uma estrutura dimensional consistente, validada e pronta para consumo no PostgreSQL e no Power BI.

4. Camada de Apresentação

4.1 Dashboard

A camada de apresentação foi desenvolvida no Power BI, conectada diretamente ao banco PostgreSQL (Neon), realizada por meio de credenciais armazenadas em variável de ambiente (.env), garantindo segurança e evitando exposição de dados sensíveis.

A construção do dashboard seguiu a mesma lógica estrutural definida na modelagem dimensional, garantindo consistência entre as camadas de dados e visualização.

A solução foi organizada em quatro páginas principais, cada uma representando uma visão específica do negócio: Sales Performance, Product Analysis, Customer & Order e Logistics. Essas páginas foram estruturadas de forma complementar, permitindo uma leitura progressiva do desempenho da operação, desde indicadores gerais até análises operacionais mais detalhadas.

O dashboard foi construído em cima de algumas perguntas de negócio, criadas para direcionar a história e construção do projeto. São elas:

- Quais mercados e regiões apresentam maior receita e rentabilidade?
- Quais categorias e produtos possuem melhor desempenho financeiro?
- Qual o impacto dos descontos sobre a margem?
- Existe relação entre volume de vendas e lucratividade?
- Quais clientes concentram maior receita?
- Como o tempo de entrega impacta a performance?

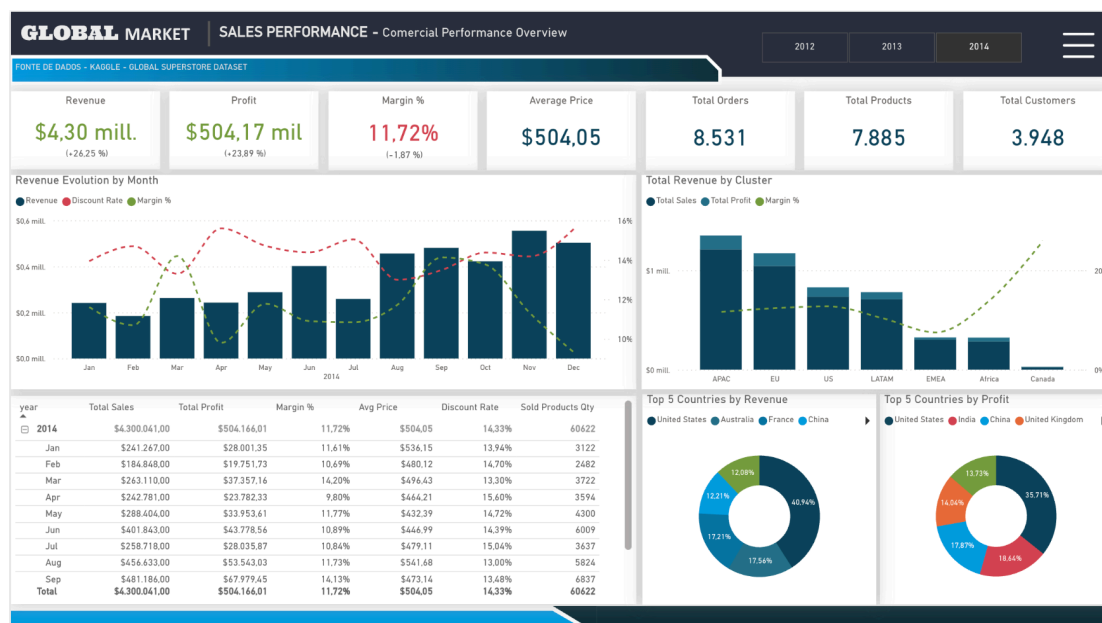
- Quais modos de envio são mais eficientes em termos de custo e margem?
- Existe sazonalidade nas vendas?
- O custo logístico compromete a rentabilidade em determinados cenários?

4.1.1 Visão Estratégica – Sales Performance

A página Sales Performance consolida os principais indicadores de desempenho da operação, permitindo uma leitura executiva do negócio.

Os principais indicadores apresentados incluem:

- Revenue (receita total)
- Profit (lucro total)
- Margin % (margem percentual)
- Average Price (ticket médio)
- Total Orders (quantidade de pedidos)
- Total Products e Total Customers



Adicionalmente, foram utilizados:

- Gráfico de evolução temporal (Revenue, Discount Rate e Margin % por mês)
- Análise por cluster (mercados)
- Distribuição por países (Top 5 Revenue e Profit)

Essa estrutura permite identificar tendências, sazonalidade e variações de desempenho ao longo do tempo.

4.1.2 Visão Tática – Product Analysis

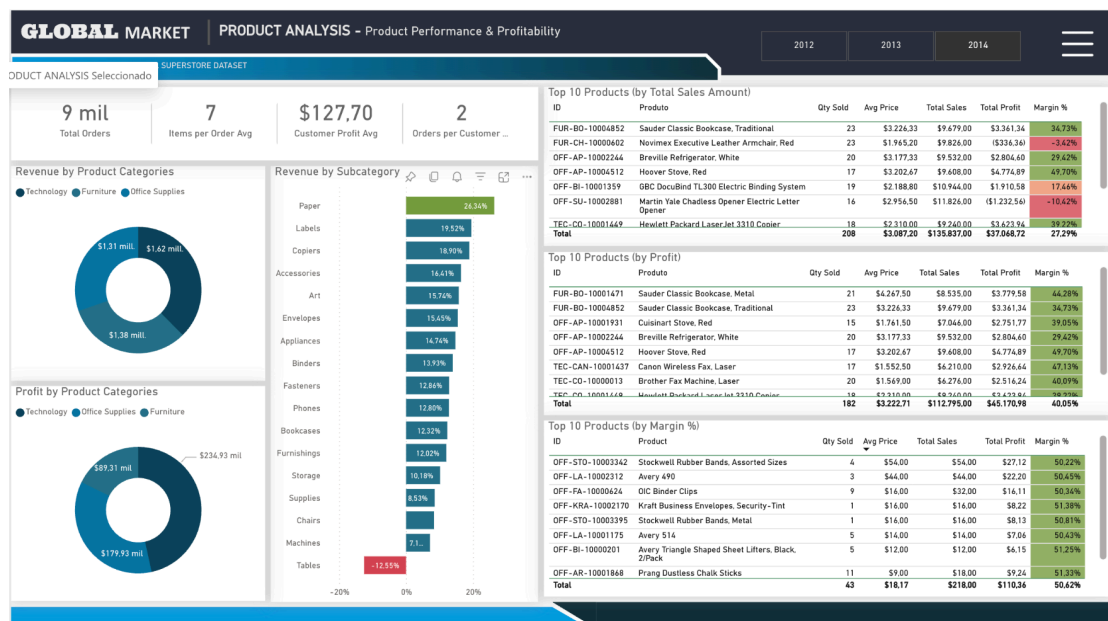
A página Product Analysis aprofunda a análise do portfólio de produtos, com foco em rentabilidade e eficiência comercial.

Os principais indicadores incluem:

- Total Orders
- Items per Order Avg
- Customer Profit Avg
- Orders per Customer

As análises foram estruturadas por:

- Categoria e Subcategoria de produto
- Ranking de produtos por Sales, Profit e Margin %



As principais visualizações utilizadas foram:

- Gráficos de rosca para distribuição de receita e lucro por categoria
- Barras horizontais para sub categorias
- Tabelas analíticas com destaque condicional de margem

Essa página permite identificar:

- Produtos mais rentáveis
- Produtos com margem negativa

- Concentração de receita por categoria

4.1.3 Visão Tática/Operacional – Customer & Order

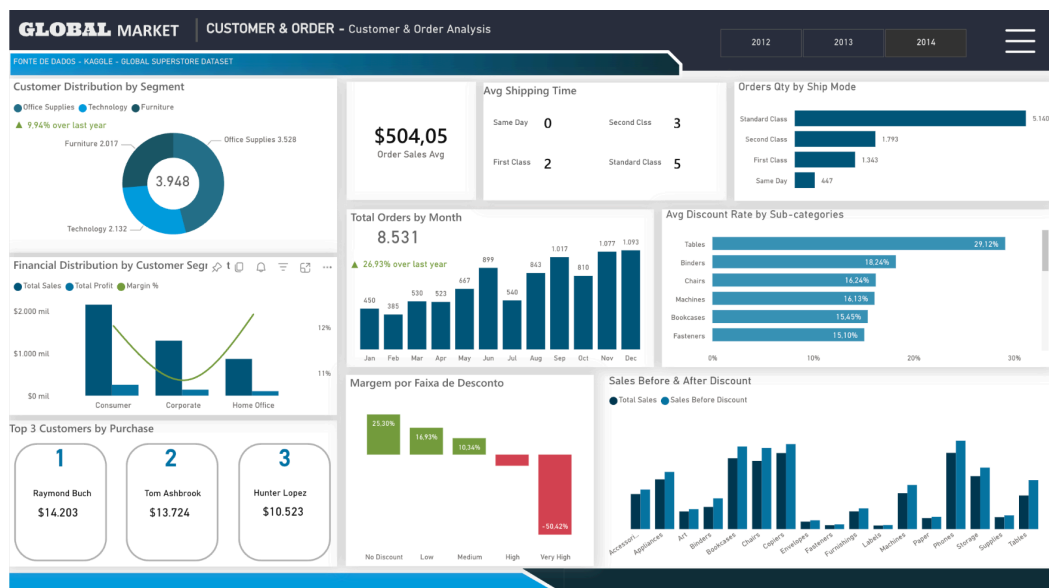
A página Customer & Order analisa o comportamento de consumo e padrões de compra.

Os principais indicadores incluem:

- Order Sales Avg
- Avg Shipping Time
- Orders by Ship Mode
- Sales After Discount

As análises contemplam:

- Distribuição de clientes por segmento
- Evolução de pedidos ao longo do tempo
- Impacto de desconto na margem
- Top clientes por volume de compra



As visualizações incluem:

- Gráficos de barras e colunas
- Distribuições percentuais
- Indicadores de variação temporal

Essa página permite compreender:

- Perfil de consumo

- Impacto de descontos na rentabilidade
- Concentração de receita em clientes

4.1.4 Visão Operacional – Logistics

A página Logistics foca na eficiência logística e seu impacto financeiro. Os principais indicadores incluem:

- Avg Shipping Time
- Total Shipping Cost
- Shipping Cost % of Revenue
- Avg Shipping Cost
- Shipping Time



As análises incluem:

- Relação entre receita e custo logístico
- Margem por modo de envio
- Volume de envios por dia da semana

Foi utilizada a seguinte medida para análise por data de envio:

```
Shipments by Ship Date =
CALCULATE(
    DISTINCTCOUNT(f_sales_order_line[order_id_bk]),
    USERELATIONSHIP(
        f_sales_order_line[ship_date_sk],
        d_calendar[date_sk]
    )
)
```

Essa abordagem permite analisar corretamente o fluxo logístico considerando a data de envio, e não apenas a data do pedido.

4.1.5 Filtros e Interatividade

O dashboard permite filtragem por:

- Ano (2012, 2013, 2014)
- Cluster - Mercado
- Categoria
- Subcategoria
- Região
- País
- Cliente ID
- Modo de Envio

A navegação entre páginas permite análise integrada entre dimensões comerciais e operacionais. Os filtros foram distribuídos de acordo com a página de análise, não sendo apresentados de forma homogênea.

4.2 Análises avançadas

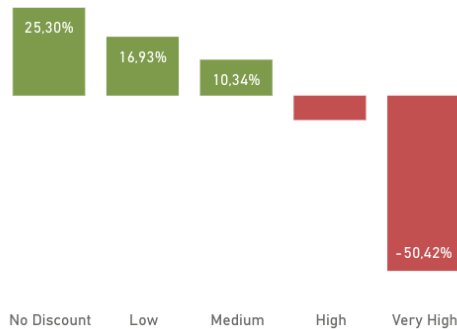
As análises avançadas foram desenvolvidas com base nas variáveis derivadas criadas no ETL, permitindo segmentações analíticas relevantes.

Foram explorados principalmente:

- Padrões de margem por faixa de desconto
- Relação entre tempo de envio e lucratividade
- Identificação de produtos com comportamento atípico (alta venda e baixa margem)

As variáveis `discount_band` e `profitability_band` permitiram análises comparativas diretas, reduzindo a complexidade de interpretação de variáveis contínuas.

Margin by Discount Range



5. Registros de Homologação

A homologação da solução foi realizada por meio da comparação entre os dados persistidos no PostgreSQL e os indicadores exibidos no Power BI. Além das métricas agregadas, foi validada a preservação da granularidade ao nível de item de pedido, garantindo que não houve duplicidade ou perda de registros durante o processo de modelagem.

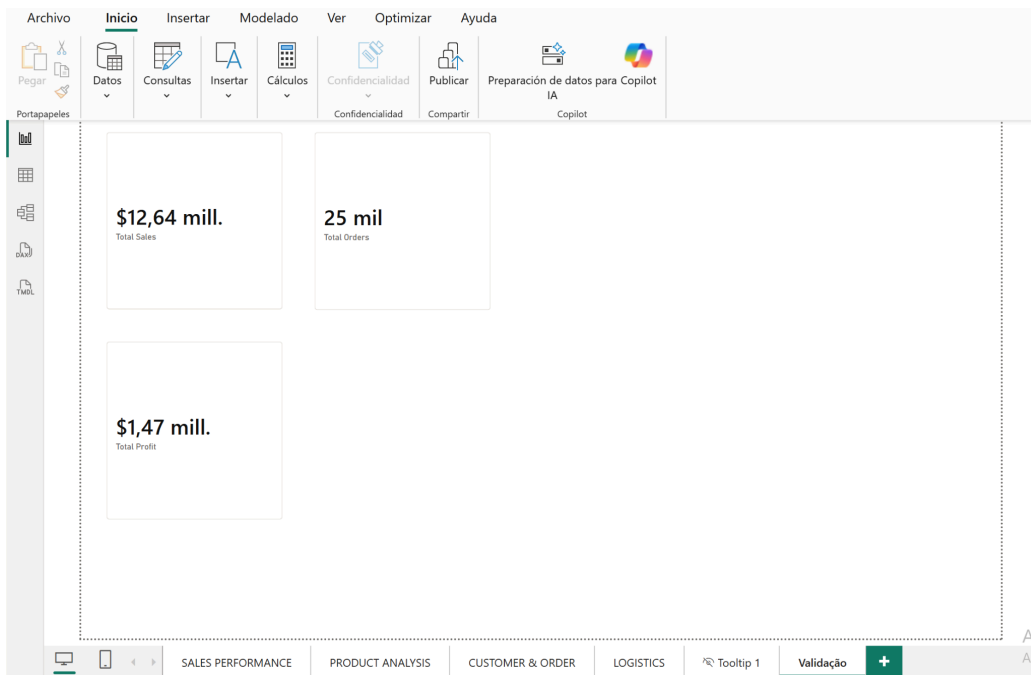
As validações foram executadas com consultas SQL diretamente na tabela fato:

```
C:\Users\paulafreire\Documents\tcc_analytics_bi>python src\03_validate_neon.py
VALIDAÇÃO ANALÍTICA DA FATO

total_linhas_fato: 51290
total_vendas: 12642905.00
total_lucro: 1467456.52
total_quantidade: 178312
total_pedidos_distintos: 25035

C:\Users\paulafreire\Documents\tcc_analytics_bi>
```

Esses valores foram comparados com os indicadores do dashboard, confirmando a consistência entre as camadas de dados e visualização.



6. Conclusões

A análise integrada dos dados evidenciou que:

- Categorias com alto volume de vendas não necessariamente apresentam maior margem, evidenciando a influência direta de descontos e custos associados.
- Descontos elevados impactam diretamente a margem. A análise de impacto do desconto foi realizada por meio da comparação entre vendas antes e depois da aplicação de desconto, permitindo avaliar a diferença absoluta e relativa por categoria.
- Os custos logísticos representam, em média, aproximadamente 10% da receita, indicando um impacto moderado sobre o resultado financeiro. No entanto, a análise integrada demonstra que seu efeito torna-se mais significativo quando combinado com altos níveis de desconto, contribuindo para a redução da margem em determinados cenários.
- Existem variações significativas entre mercados e categorias

A decisão de concentrar a lógica analítica no ETL, ao invés da camada de visualização, contribuiu para maior consistência, rastreabilidade e desempenho do

modelo. A estrutura dimensional permitiu identificar essas relações de forma consistente e rastreável.

Com base nos achados, recomenda-se:

- Revisão da política de descontos em produtos com margem negativa
- Otimização de modos de envio com alto custo relativo
- Priorização de categorias com maior retorno financeiro
- Monitoramento de clientes com alta concentração de receita

Dentro das lições aprendidas durante o desenvolvimento do projeto, destacam-se:

- A importância de estruturar corretamente a camada analítica antes da visualização
- O papel do ETL na garantia de consistência dos dados
- A relevância da modelagem dimensional para análise em BI

Entre as limitações do projeto:

- Uso de dados históricos sem atualização em tempo real
- Ausência de variáveis externas (ex.: concorrência, economia)

Possíveis extensões incluem:

- Aplicação de modelos preditivos (Machine Learning)
- Integração com dados em tempo real
- Evolução para análises de forecast e churn

7. Links

1. Conjunto de Dados (*Global Superstore Dataset*):

<https://www.kaggle.com/datasets/fatihilhan/global-superstore-dataset/data/code>

2. ChartDB - Estruturação da Modelagem Dimensional (necessário ser membro para acessar o link):

<https://app.chartdb.io/diagram/ccf20fd2a1504336858f65>

3. Banco de Nuvem Neon:

<https://console.neon.tech/app/projects/proud-water-02367062>